

UNIVERSITY OF SCIENCE – VNU-HCM

Course: Mobile Programming

PROJECT PROPOSAL

IMPLEMENTING CLEAN ARCHITECTURE IN ANDROID PROJECTS

FieldFlow

Offline-First Inspection and Maintenance Application

Circuit-Based Feature-Modular Clean Architecture

Team: [Enter team name]

Members: Huy, Thắng, Linh, Lĩnh

Seminar topic: Topic 11 – Clean Architecture in Android

Date: July 19, 2026

Contents

1	Proposal Summary	1
2	Seminar Goals and Architectural Thesis	1
3	Product Idea: FieldFlow	2
3.1	What exactly is the application?	2
3.2	A complete usage scenario	3
4	Why Is FieldFlow a Strong Clean Architecture Case Study?	4
5	Functional Scope	4
5.1	Required seminar version	4
5.2	Required business rules	5
5.3	Outside the initial scope	5
6	Selected Architecture	5
6.1	Precise name and description	5
6.2	Architecture overview	6
6.3	Responsibilities and forbidden dependencies	6
7	Modern Presentation with Slack Circuit	7
7.1	Why not rely only on conventional MVVM?	7
7.2	Unidirectional Data Flow	7
7.3	Inspection workflow state model	7
7.4	Illustrative contract	8
8	Domain Layer: Where Clean Architecture Is Most Visible	8
8.1	Domain models	8
8.2	Core Use Cases	9
8.3	Illustrative validation rule	9
8.4	Repository and output ports	10
9	Data Layer: Offline-First, Source of Truth, and Adapters	10
9.1	The local database as the canonical source of truth	10
9.2	Planned adapters	10
9.3	Offline writes and synchronization queue	11
10	Feature-First Multi-Module Design	11
10.1	Proposed module structure	11
10.2	Dependency rules	11
11	Why Is This Design Modern, Clean, and Strong?	12
12	Demonstrating the Core Benefits	12
12.1	Separation of Concerns	12
12.2	Testability	13
12.3	Maintainability	13

12.4 Scalability	13
12.5 Resilience	13
13 Seminar Demonstration Scenarios	13
13.1 Demo 1: Circuit workflow and immutable state	13
13.2 Demo 2: Independent Domain validation	13
13.3 Demo 3: Process recovery	14
13.4 Demo 4: Offline-first behavior and synchronization state	14
13.5 Demo 5: Hot-swapping the repository	14
13.6 Demo 6: Hot-swapping the report adapter	14
13.7 Demo 7: Compile-time boundary	14
14 Comparison with Alternative Approaches	14
15 Proposed Technology Stack	15
16 Five-Week Implementation Roadmap	16
17 Four-Person Team Allocation	16
18 Definition of Done	16
19 Risks and Trade-Offs	17
20 Sources and Rationale	18
21 Conclusion	18

1. Proposal Summary

The team proposes **FieldFlow**, an offline-first Android application for facility, equipment, inspection, and maintenance workflows. Users can register assets, build checklists, perform multi-step inspections, record failures, attach evidence, create maintenance issues, save drafts, recover work after the application is closed, and export reports.

The selected design is **Circuit-Based Feature-Modular Clean Architecture**. This is not merely a modern-sounding label. It is an intentional combination of complementary architectural decisions:

- **Clean Architecture** is the primary system architecture. It protects business rules and governs dependency direction.
- **Slack Circuit** implements the Presentation Layer through **Screen**, **Presenter**, immutable **UiState**, **UiEvent**, and **Compose Ui**.
- **Feature-first multi-module design** turns conceptual boundaries into Gradle-level dependency rules.
- **Repository ports and replaceable adapters** isolate the Domain from Room, file storage, report export, fake data, and network synchronization.
- **An offline-first local source of truth** allows the core workflow to remain available without an Internet connection.
- **State-driven workflows** make inspection steps, validation, confirmation dialogs, recovery, and navigation explicit and testable.

The proposal places Clean Architecture at the center. Frameworks, the UI toolkit, database, camera, files, and network are treated as outer details. Inspection and maintenance policies remain in a pure Kotlin Domain that can be tested independently and is not locked to a specific Android technology.

2. Seminar Goals and Architectural Thesis

The group's topic is **Implementing Clean Architecture in Android Projects**. Therefore, the product is not the final objective; it is a case study used to demonstrate an architectural decision through working code.

The seminar should answer four questions:

1. Where do architectural problems appear as an Android application grows?
2. Why is a conventional Presentation pattern alone insufficient to protect an entire system?
3. How can Clean Architecture be implemented in a modern Compose-based Android application?
4. How can separation of concerns, testability, maintainability, scalability, and resilience be demonstrated through real code?

The central thesis is:

A modern architecture does not need to reject proven principles. For this seminar, the strongest design is to use Clean Architecture to protect the business core, implement

Presentation with the modern Compose-first Slack Circuit framework, enforce boundaries with Gradle modules, and build the Data Layer around an offline-first strategy.

Slack describes Circuit as a lightweight, extensible framework built with Compose and used in production at Slack [1]. Clean Architecture contributes the Dependency Rule: source-code dependencies should point inward toward policy and business rules [15].

3. Product Idea: FieldFlow

3.1. What exactly is the application?

FieldFlow is a **standalone Android inspection and maintenance application**. The seminar version does not require a real school, company, organization, or production backend. Users may create their own records or select deterministic sample data.

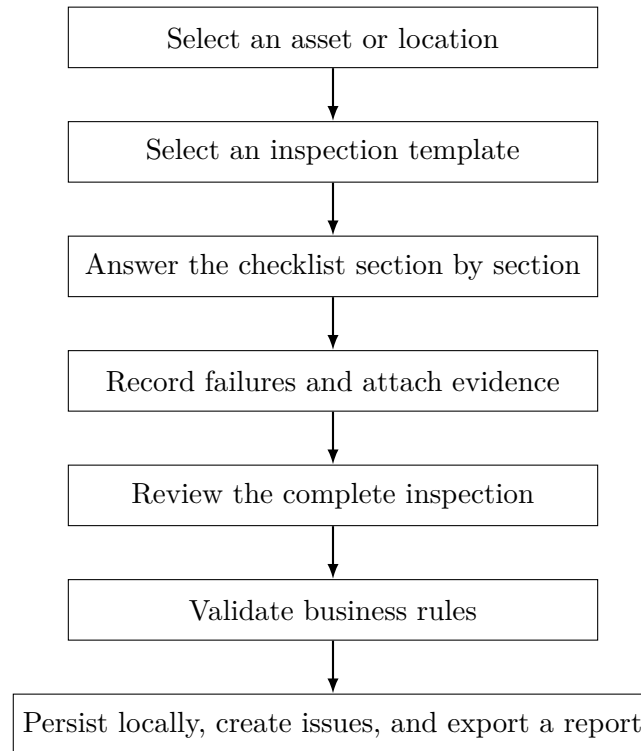
Example demo data:

Object	Inspection type	Status
Computer Lab I.44	Computer Lab Inspection	In progress
Projector P-204	Equipment Check	Due soon
Laboratory A2	Safety Inspection	Critical failure

Users can:

- create locations, assets, and equipment;
- create or select an inspection template;
- complete a checklist section by section;
- mark an item as **Pass**, **Fail**, or **NotApplicable**, or enter a measured value;
- add notes and photographic evidence;
- create a maintenance issue when a failure is discovered;
- save a draft and resume the inspection later;
- complete an inspection and export a report;
- observe synchronization states such as **Pending**, **Syncing**, **Synced**, and **Failed**.

3.2. A complete usage scenario



This process contains multiple states, branching failures, and several external boundaries. Architecture is therefore not decorative; it becomes a practical tool for controlling real complexity.

4. Why Is FieldFlow a Strong Clean Architecture Case Study?

Table 1: Product problems and the architectural mechanisms that address them

FieldFlow problem	Architectural mechanism
Multi-step workflow with backward navigation	Circuit navigation, Presenter, and immutable state
Business-specific validation	Pure Kotlin Domain models and Use Cases
Images, files, and PDF output	Ports and Adapters; framework details remain outside the Domain
Network loss or process termination	Room source of truth, draft persistence, and process recovery
Multiple checklist types	Polymorphic domain models and validation strategies
Creating an issue from a failed item	Use Case orchestration and transaction boundaries
Switching among local, fake, and remote sources	Repository ports and DI bindings
Several relatively independent features	Feature modules and compile-time visibility
Demonstrating testability	Fake repositories, Circuit testing APIs, and Domain unit tests

Compared with a simple Movie application that only loads and displays a list, FieldFlow has clearer business rules and state transitions. Compared with a full e-commerce system, it avoids real payment and a complex production backend. It is professional enough for architecture to create visible value while remaining feasible for a student team.

5. Functional Scope

5.1. Required seminar version

1. **Dashboard:** inspections in progress, unresolved issues, recent inspections, and synchronization state.
2. **Assets:** locations, rooms, equipment, and next inspection dates.
3. **Templates:** section-based checklists with several item types.
4. **Inspection Workflow:** start, pause, resume, review, and complete an inspection.
5. **Evidence:** notes and photographs selected from the device.
6. **Issues:** create, update, and track maintenance issues.

7. **Reports:** generate a summary report and export PDF or JSON.
8. **Offline Storage:** Room is the canonical data source.
9. **Fake Data Mode:** run the complete application with deterministic fake adapters for demos and tests.
10. **Recovery:** close the application during an inspection and reopen the same draft at the correct step.

5.2. Required business rules

- An inspection cannot be completed while a required item is unanswered.
- A **Critical** + **Fail** result requires evidence and must create a maintenance issue.
- An inspection session may only move through valid lifecycle transitions.
- The inspection score is weight-based and excludes **NotApplicable** items.
- A report may only be generated after the inspection passes validation.
- The next inspection date is calculated from the template or asset recurrence policy.

5.3. Outside the initial scope

The team does not need to implement enterprise accounts, advanced role-based authorization, production push notifications, GPS tracking, a real QR scanner, a production cloud backend, or AI image analysis. These may become future adapters, but they are not required to prove the architecture.

6. Selected Architecture

6.1. Precise name and description

Circuit-Based Feature-Modular Clean Architecture

The precise description is:

Clean Architecture is the primary architecture of the system. Slack Circuit implements the Presentation Layer in a Compose-first, Unidirectional Data Flow style. The codebase is organized into feature modules. The Domain is pure Kotlin and owns business rules and repository ports. The Data Layer provides replaceable adapters and uses a local database as the source of truth.

Circuit does not replace Clean Architecture. Circuit addresses Presentation concerns: state, events, navigation, overlays, retention, and testing. Clean Architecture continues to govern Domain policies, Use Cases, repository boundaries, data mapping, and dependency direction.

6.2. Architecture overview

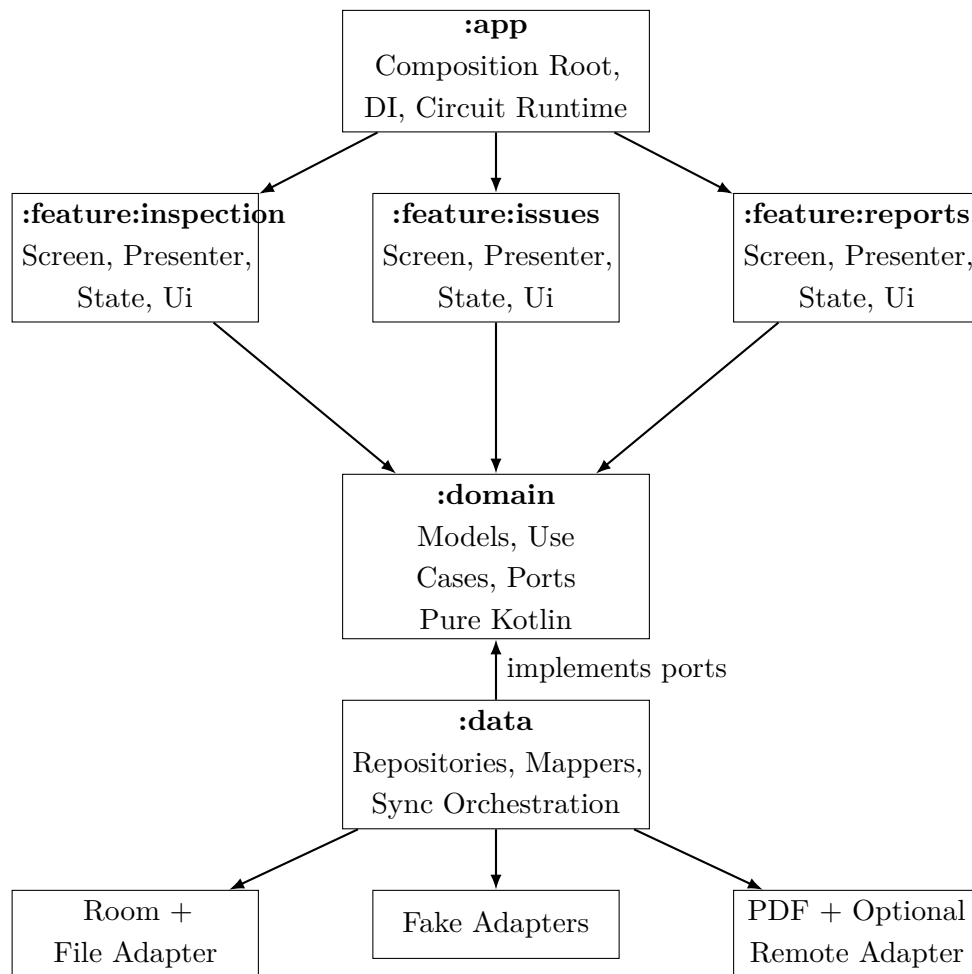


Figure 1: Architecture overview and dependency direction toward the Domain

The central rule is:

$$\text{Feature} \rightarrow \text{Domain} \leftarrow \text{Data}$$

The Domain does not import Android, Compose, Circuit, Room, WorkManager, a PDF library, or a file API. Those technologies are details in the outer layers.

6.3. Responsibilities and forbidden dependencies

Area	Responsibility	Must not know
Circuit Ui Presenter	Render state and emit events Produce UiState; coordinate interaction and navigation	Repository, DAO, validation rules SQL, file paths, PDF implementation
Domain	Business rules, Use Cases, ports	Android, Room, Circuit, Compose
Data	Implement ports, map data, manage the source of truth	UI and navigation details
App/Framework	DI, workers, file chooser, operating-system integration	Internal business rules

7. Modern Presentation with Slack Circuit

7.1. Why not rely only on conventional MVVM?

MVVM can organize UI code effectively. However, when the architecture is reduced to one `ViewModel` per screen, classes can accumulate state, navigation, validation, side effects, and data orchestration. Circuit provides a more explicit contract between presentation logic and UI.

Its principal components are:

- **Screen:** identifies a destination and its navigation parameters;
- **Presenter:** produces state, handles events, calls Use Cases, and navigates;
- **UiState:** an immutable snapshot of everything the UI needs to render;
- **UiEvent:** a user action sent back to presentation logic;
- **Ui:** a pure Composable that contains no business policy.

Circuit keeps Presenter and Ui from directly accessing each other; they communicate only through state and events. Both are Compose-based, but the Presenter uses the Compose runtime to produce state without emitting UI [1, 2, 3].

7.2. Unidirectional Data Flow

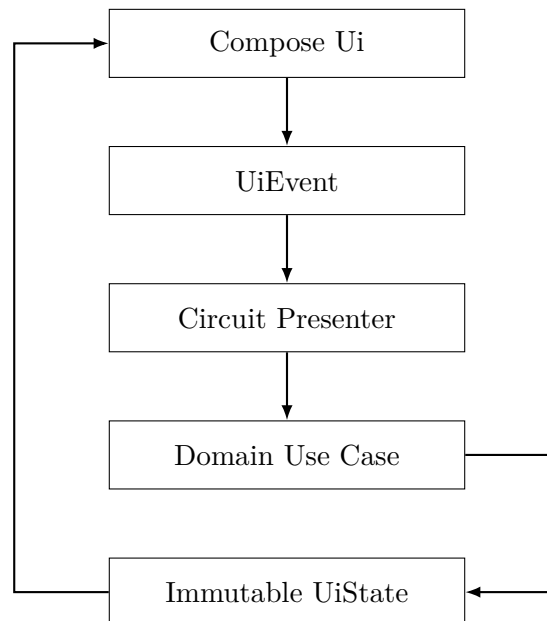


Figure 2: State flows downward and events flow upward

7.3. Inspection workflow state model

An inspection session requires more than Loading and Success. Representative states are:

LoadingTemplate → Editing → Validating → Reviewing → Saving → Completed

Additional branches include:

EvidenceRequired, ValidationFailed, DraftRecovered, SyncPending, Error

Circuit navigation manages typed `Screen` objects in the back stack. Overlays suit dialogs and bottom sheets that return typed results, such as confirmation before completion or severity selection [4, 5].

7.4. Illustrative contract

```
@Parcelize
data class InspectionScreen(
    val inspectionId: InspectionId
) : Screen {
    sealed interface State : CircuitUiState {
        data object Loading : State

        data class Editing(
            val section: InspectionSectionUi,
            val progress: InspectionProgress,
            val canContinue: Boolean,
            val eventSink: (Event) -> Unit
        ) : State

        data class Review(
            val summary: InspectionSummaryUi,
            val eventSink: (Event) -> Unit
        ) : State
    }

    sealed interface Event : CircuitUiEvent {
        data class AnswerChanged(
            val itemId: ItemId,
            val answer: InspectionAnswer
        ) : Event
        data object NextSection : Event
        data object SaveDraft : Event
        data object CompleteRequested : Event
    }
}
```

The UI does not know where the checklist is stored. The Presenter does not know the database tables. The Domain does not know which dialog or Compose component is visible.

8. Domain Layer: Where Clean Architecture Is Most Visible

8.1. Domain models

The main models include:

- Asset, Location, and InspectionTemplate;
- InspectionSection, ChecklistItem, and InspectionAnswer;

- `InspectionSession` and `InspectionStatus`;
- `Evidence`, `MaintenanceIssue`, and `IssueSeverity`;
- `InspectionScore`, `ValidationError`, and `NextInspectionDate`.

These models contain no Room annotations, UI `Parcelable` details, or concrete file-system paths.

8.2. Core Use Cases

Use Case	Responsibility
<code>StartInspection</code> UseCase	Create a valid inspection session from an asset and a template.
<code>SaveInspectionDraft</code> UseCase	Persist progress, answers, and the current section.
<code>ValidateInspection</code> UseCase	Return a structured list of business-rule violations.
<code>CompleteInspection</code> UseCase	Validate, calculate the score, create required issues, and complete the transaction.
<code>CreateMaintenanceIssue</code> UseCase	Create an issue from a failed item and severity.
<code>CalculateInspectionScore</code> UseCase	Calculate a weighted score while excluding <code>NotApplicable</code> items.
<code>ScheduleNextInspection</code> UseCase	Calculate the next inspection date from the recurrence policy.
<code>GenerateInspectionReport</code> UseCase	Produce a report model independent of any PDF library.

8.3. Illustrative validation rule

```
class ValidateInspectionUseCase {
    operator fun invoke(
        session: InspectionSession,
        template: InspectionTemplate
    ): List<InspectionValidationError> = buildList {
        template.requiredItems.forEach { item ->
            if (session.answerFor(item.id) == null) {
                add(RequiredItemUnanswered(item.id))
            }
        }

        session.failedCriticalAnswers.forEach { answer ->
            if (answer.evidenceIds.isEmpty()) {
                add(CriticalFailureNeedsEvidence(answer.itemId))
            }
        }
    }
}
```

This Use Case can run through JUnit on the JVM without an emulator, Activity, Room, or the Circuit runtime.

8.4. Repository and output ports

```
interface InspectionRepository {
    fun observeInspection(id: InspectionId): Flow<InspectionSession>
    suspend fun saveDraft(session: InspectionSession)
    suspend fun complete(result: CompletedInspection)
}

interface EvidenceStore {
    suspend fun persist(source: EvidenceSource): EvidenceReference
    suspend fun delete(reference: EvidenceReference)
}

interface ReportExporter {
    suspend fun export(report: InspectionReport): ExportedReport
}
```

The Domain defines what the system needs. Data and framework modules determine how Room is used, how images are copied, and how a PDF is generated.

9. Data Layer: Offline-First, Source of Truth, and Adapters

9.1. The local database as the canonical source of truth

FieldFlow should operate in computer labs, basements, and other places with unreliable connectivity. Room is the canonical source observed by upper layers. Android Architecture Guidance presents Single Source of Truth and UDF as principles for consistent data; in an offline-first application, a database commonly serves as the source of truth [8, 11, 12].

Read flow:

Room → Repository Flow → Use Case → Presenter → UiState → Ui

Write flow:

UiEvent → Presenter → Use Case → Room transaction → New Flow emission

The UI does not read simultaneously from database and network and then decide which value is correct.

9.2. Planned adapters

- RoomInspectionRepository: production repository on the device;
- FakeInspectionRepository: deterministic data for demos and tests;
- InMemoryInspectionRepository: fast isolated testing;
- AndroidEvidenceStore: copies and stores images in app-managed storage;

- **PdfReportExporter**: converts the report model to PDF;
- **JsonReportExporter**: a replaceable adapter that demonstrates the boundary;
- **FakeRemoteSyncAdapter**: simulates success, delay, and failure;
- **WorkManagerSyncScheduler**: framework adapter for a future queued synchronization mechanism.

9.3. Offline writes and synchronization queue

The seminar version can persist locally first and then create a synchronization command in the **Pending** state. A fake remote adapter enables deterministic demonstrations:

Pending → Syncing → Synced

or:

Pending → Syncing → Failed → Retry

The remote adapter remains an implementation detail. Even without a real backend, the architecture and synchronization state can be demonstrated through a deterministic fake.

10. Feature-First Multi-Module Design

10.1. Proposed module structure

```
:app
:domain
:data
:core:model
:core:database
:core:designsystem
:core:testing
:feature:dashboard
:feature:assets
:feature:templates
:feature:inspection
:feature:issues
:feature:reports
```

10.2. Dependency rules

- A feature depends on the Domain and only the core modules it requires.
- Data depends on the Domain in order to implement its ports.
- The Domain is a Kotlin module and, where practical, does not use the Android Gradle plugin.
- A feature does not depend directly on a DAO, file implementation, or exporter implementation.
- The app module is the Composition Root, where Circuit, navigation, and the DI graph are assembled.
- Android-specific adapters remain in the outermost layer.

Android Developers describes modularization as organizing a codebase into loosely coupled, self-contained parts with clear purposes, while warning against excessive module splitting when overhead exceeds benefit [13]. The team therefore creates modules only for features with meaningful boundaries rather than creating one module per class.

11. Why Is This Design Modern, Clean, and Strong?

Table 3: Every decision has a concrete technical purpose

Decision	Value
Clean Architecture	Business rules remain independent from frameworks, databases, and UI.
Slack Circuit	Compose-first Presentation with explicit Screen contracts, state/events, and integrated navigation.
Immutable UiState + UDF	State transitions are easier to trace, replay, and test.
Feature modules	Gradle enforces boundaries and visibility.
Pure Kotlin Domain	Fast unit tests without an Android runtime.
Repository ports	Replace local, fake, or remote implementations without changing callers.
Local SSOT	Stable offline behavior and one consistent data path.
File/PDF/sync adapters	Android SDKs and third-party libraries do not leak into the Domain.
Composition Root	Concrete implementations are assembled only at the boundary.

Modernity does not come from creating as many layers as possible. It comes from combining the stable principles of Clean Architecture with tools suited to the Compose era: Circuit, immutable state, typed screens, feature modules, a local source of truth, and first-party testing utilities.

12. Demonstrating the Core Benefits

12.1. Separation of Concerns

- Compose Ui renders state and emits events.
- Circuit Presenter coordinates screen state and interaction.
- Domain Use Cases execute business rules.
- Repository ports describe boundaries.
- Data adapters handle Room, files, synchronization, and mapping.
- The App/Framework layer integrates the Android operating system.

12.2. Testability

Circuit provides testing artifacts such as `presenterTestOf()`, `Presenter.test()`, `FakeNavigator`, and `TestEventSink`. These utilities support assertions over state emissions, navigation, and incoming UI events [6]. The team can build a testing pyramid containing:

- Domain unit tests for validation, scoring, and lifecycle transitions;
- Presenter tests with fake repositories and a fake navigator;
- UI tests using predefined states;
- Repository integration tests with in-memory Room;
- a limited number of end-to-end tests for the most important vertical slice.

12.3. Maintainability

Changing the PDF library should require changes only in `PdfReportExporter`. Changing the Room schema should remain mainly within data/database modules. Changing the UI style should remain within feature and design-system modules. When contracts remain stable, unrelated Domain and feature code should not change transitively.

12.4. Scalability

New capabilities such as QR asset lookup, cloud synchronization, or recurring inspection policies can be introduced through dedicated adapters or modules. Team ownership can be assigned by feature instead of forcing everyone to edit a monolithic `:app` module.

12.5. Resilience

Drafts are persisted locally, state can be recovered, queued synchronization has explicit states, and core data does not depend on the network. These qualities are difficult to demonstrate in a simple “load and display” application.

13. Seminar Demonstration Scenarios

13.1. Demo 1: Circuit workflow and immutable state

Start an inspection, move through sections, open a severity-selection overlay, return to an earlier step, and observe state changes. Identify `Screen`, `Presenter`, `UiState`, `UiEvent`, and `Compose Ui` in the code.

13.2. Demo 2: Independent Domain validation

Attempt to complete an inspection while a required question is unanswered or a critical failure lacks evidence. The application rejects the operation with a structured error. Then run the unit test for `ValidateInspectionUseCase`.

13.3. Demo 3: Process recovery

Close the application in the middle of an inspection. Reopen it and continue from the same draft and section. This proves that important state is not held temporarily inside UI components.

13.4. Demo 4: Offline-first behavior and synchronization state

Simulate network loss, complete an inspection, and create an issue. The data remains locally available with the `Pending` state. Configure the fake remote adapter to succeed and trigger a retry so the state becomes `Synced`.

13.5. Demo 5: Hot-swapping the repository

Change the binding:

```
bind<InspectionRepository>()
    .to<RoomInspectionRepository>()
```

to:

```
bind<InspectionRepository>()
    .to<FakeInspectionRepository>()
```

The application still runs without changes to Circuit Ui, Presenter, or Domain Use Cases. This is a visible demonstration of Dependency Inversion.

13.6. Demo 6: Hot-swapping the report adapter

Replace `PdfReportExporter` with `JsonReportExporter`. The Domain only produces an `InspectionReport`; the output format is an outer implementation detail.

13.7. Demo 7: Compile-time boundary

Show that the Domain declares no dependency on Circuit, Compose, or Room. An intentional framework import in the Domain should fail to build. Multi-module design turns the Dependency Rule from a convention into a technical constraint.

14. Comparison with Alternative Approaches

Approach	Strength	Why it is not the complete primary design
Conventional MVVM	Popular, approachable, and compatible with Android View-Model	Primarily addresses UI concerns; whole-system boundaries may remain weak when separated only by packages.
Pure MVI reducer	Clear and deterministic state transitions	Does not independently solve repository, storage, file, and module boundaries.

Approach	Strength	Why it is not the complete primary design
Square Workflow	Powerful for state machines and nested workflows	Its learning curve and framework scope exceed the team's immediate needs.
Molecule	Uses the Compose runtime to produce StateFlow	A state-production tool rather than a complete application architecture.
Decompose	Strong component lifecycle and navigation, especially for KMP	Kotlin Multiplatform is not the focus of this Android seminar.
Slack Circuit	Compose-first Presenter/Ui, navigation, overlays, and testing	Selected for Presentation, but Clean Architecture is still required for Domain and Data.
Clean Architecture with only three packages	Easy to start	Does not enforce boundaries and does not use a modern Presentation framework.

Circuit offers a strong balance of modernity, official documentation, production experience, demonstrability, and implementation scope. Clean Architecture prevents the system from becoming excessively coupled to Circuit.

15. Proposed Technology Stack

Component	Selection and role
Language	Kotlin
UI	Jetpack Compose + Material 3
Presentation architecture	Slack Circuit
Asynchronous streams	Kotlin Coroutines and Flow
Local database	Room
Dependency Injection	Hilt/Dagger or Metro; the team selects one and uses it consistently
Background queue	WorkManager in the framework layer when synchronization is demonstrated
Serialization	Kotlin Serialization
Report export	Android PDF API or another library placed behind the <code>ReportExporter</code> port
Testing	JUnit, Coroutines Test, Turbine, and Circuit testing utilities
Build organization	Gradle multi-module project + version catalog
CI	GitHub Actions for build, lint, and unit tests

A network library should not be added unless the team implements a remote adapter. Clean design also means avoiding dependencies that provide no demonstrated value.

16. Five-Week Implementation Roadmap

Week	Objective	Required result
1	Architecture skeleton	Freeze Domain contracts and the module graph; set up Circuit, the DI composition root, fake repositories, and CI.
2	Domain and Data core	Models, validation rules, Use Cases, Room schema, mappers, evidence storage, and unit tests.
3	Inspection vertical slice	Assets, templates, and the inspection workflow through Screen–Presenter–UiState–Ui.
4	Issues, reports, and recovery	Issue lifecycle, report model/export adapter, draft recovery, error/empty states, and Presentation tests.
5	Freeze and seminar	Offline/sync simulation, hot-swap demonstration, boundary review, slides, rehearsal, and documentation.

The first-week vertical slice should run from fake data to UI so the team can validate Circuit and the module graph before expanding the product.

17. Four-Person Team Allocation

Member	Primary responsibility
Member 1	Architecture owner: module graph, Circuit setup, DI composition root, dependency review, and integration.
Member 2	Domain owner: models, validation, score, lifecycle, Use Cases, and unit tests.
Member 3	Data owner: Room, repositories, mappers, evidence storage, fake synchronization, and basic migrations.
Member 4	Feature/UI owner: Compose UIs, Circuit contracts, navigation, overlays, report UI, and demonstration flow.

Every important feature should be reviewed by at least two people. Architecture must not become knowledge held only by the architecture owner.

18. Definition of Done

The project is complete only when:

- the Domain has no Android or framework dependency;
- the inspection workflow clearly contains Screen, Presenter, UiState/Event, and Compose Ui;
- important business rules have unit tests;
- the application can run with both Room and fake repositories;

- no feature calls a DAO, file-storage implementation, or PDF library directly;
- a draft is recovered after the application is closed and reopened;
- core data remains usable offline;
- validation, adapter replacement, and compile-time boundary demonstrations are stable;
- the team can explain the benefits, trade-offs, and situations in which the architecture should not be used.

19. Risks and Trade-Offs

Risk		Impact	Control
Circuit is new to the team		Slower initial progress	Build a small vertical slice first and follow the official tutorial.
Too many modules		More build configuration	Create modules only for features with clear boundaries and ownership.
Redundant Cases	Use	Boilerplate	Create a Use Case only for a business rule, orchestration, or reusable logic.
Overly ambitious offline synchronization		Scope drifts toward backend development	Use a deterministic fake remote adapter and explicit synchronization states.
Images and PDF export consume time		Core feature work slows down	Use a simple picker/file implementation and keep it behind a port.
Architecture	over-engineering	Effort without corresponding value	Every abstraction must support a test, demonstration, or real replacement scenario.

There is no universally best architecture. For a one-screen application with little logic, this design may be excessive. FieldFlow is selected because it has enough complexity for boundaries, state management, and offline-first design to solve genuine problems.

20. Sources and Rationale

Table 6: Source of each architectural decision

Topic	Primary source	Application in this project
Dependency Rule	Robert C. Martin, The Clean Architecture	Domain remains independent from frameworks and implementations.
Circuit Presenter/Ui	Official Slack Circuit documentation	Compose-first Presentation through state and events.
Presenter scal-ing/testing	Slack Circuit documentation	Decompose large Presenters and test state emissions.
UDF and UI state	Android Developers	State flows downward and events flow upward.
Repositories/Data Sources	Android Developers	Data access is separated from UI and Domain callers.
Offline-first	Android Developers	The local source is the canonical source of truth.
Modularization	Android Developers	Features and boundaries are represented by modules.
Reference implementation	Now in Android	A modular Kotlin/Compose application with repositories and fake implementations.

Android Architecture Guidance describes modern app architecture through layered or adaptive design, UDF, state holders, repositories, coroutines/Flow, and an optional Domain Layer when business logic becomes complex [8, 9, 10]. Slack Circuit adds a Compose-first Presentation architecture with production experience and first-party testing APIs [1, 6, 7].

21. Conclusion

FieldFlow is designed so that every architectural benefit can be demonstrated by a concrete behavior:

- the multi-step workflow demonstrates Circuit, immutable state, and typed navigation;
- validation and scoring demonstrate an independent Domain;
- Room, fake repositories, evidence storage, and exporters demonstrate Ports and Adapters;
- draft recovery and synchronization state demonstrate offline-first resilience;
- feature modules demonstrate scalability and compile-time boundaries;
- unit tests, Presenter tests, and hot-swapping demonstrate testability.

The final seminar message is:

The team is not introducing an architecture merely because it is new. It is presenting a modern way to implement Clean Architecture on Android: Slack Circuit manages Presentation in the Compose era; a pure Kotlin Domain protects business rules; Gradle

modules enforce boundaries; and the Data Layer uses a local source of truth with replaceable adapters. FieldFlow is complex enough for architecture to solve real problems, yet controlled enough for the team to implement and explain completely.

References

- [1] Slack Technologies, *Circuit – A Compose-driven architecture for Kotlin and Android*, official documentation.
<https://slackhq.github.io/circuit/>
- [2] Slack Technologies, *Circuit Presenter*.
<https://slackhq.github.io/circuit/docs/presenter/>
- [3] Slack Technologies, *Circuit: States and Events*.
<https://slackhq.github.io/circuit/getting-started/states-and-events/>
- [4] Slack Technologies, *Circuit Navigation*.
<https://slackhq.github.io/circuit/navigation/>
- [5] Slack Technologies, *Circuit Overlays*.
<https://slackhq.github.io/circuit/docs/overlays/>
- [6] Slack Technologies, *Circuit Testing*.
<https://slackhq.github.io/circuit/docs/testing/>
- [7] Slack Technologies, *Scaling Presenters*.
<https://slackhq.github.io/circuit/docs/presenter-patterns/>
- [8] Android Developers, *Guide to App Architecture*.
<https://developer.android.com/topic/architecture>
- [9] Android Developers, *Recommendations for Android Architecture*.
<https://developer.android.com/topic/architecture/recommendations>
- [10] Android Developers, *Domain Layer*.
<https://developer.android.com/topic/architecture/domain-layer>
- [11] Android Developers, *Data Layer*.
<https://developer.android.com/topic/architecture/data-layer>
- [12] Android Developers, *Build an Offline-First App*.
<https://developer.android.com/topic/architecture/data-layer/offline-first>
- [13] Android Developers, *Guide to Android App Modularization*.
<https://developer.android.com/topic/modularization>
- [14] Google, *Now in Android – Kotlin and Jetpack Compose Reference App*.
<https://github.com/android/nowinandroid>
- [15] Robert C. Martin, *The Clean Architecture*, 2012.
<https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
- [16] Robert C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.